Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ciencias de la Computación

Tarea 2
Árboles B y B^ϵ

Integrante: Matías Rojas

Profesores: Benjamín Bustos,
Gonzalo Navarro

Fecha de entrega: 25 de Octubre de 2026

Índice

1. Introducción 1
   1.1. Hipótesis 1
2. Desarrollo 2
   2.1. Estructuras de Datos y Ajuste de Bloque 2
   2.2. Decisiones de Implementación y Simulación de E/S 2
   2.3. Inserción y división de nodos 2
   2.4. Algoritmo de Vaciado de Buffers (Flush) 3
   2.5. Altura y efecto de ε 3
   2.6. Búsquedas por Rango sobre Archivo Binario 3
3. Resultados 4
   3.1. Especificaciones del Experimento 4
      3.1.1. Setup Experimental 4
   3.2. Costo de Inserción y Construcción 5
   3.3. Costos de Búsqueda por Rango 5
   3.4. Tamaño de los árboles 6
   3.5. Comparación de una misma búsqueda 6
   3.6. Resultados del Parámetro α 7
4. Análisis 7
   4.1. Conclusión 8

1. Introducción

El manejo eficiente de grandes volúmenes de datos es un desafío fundamental en las ciencias de la computación, especialmente cuando las estructuras de datos superan la capacidad de la memoria principal. En estos escenarios, el costo de las operaciones de entrada y salida (I/O) hacia y desde la memoria secundaria se convierte en el principal cuello de botella. Para mitigar esto, se utilizan estructuras de datos orientadas a memoria externa, donde el árbol B ha sido el estándar histórico gracias a su alta tasa de ramificación y al empaquetamiento de sus nodos para coincidir con el tamaño de bloque del disco.

Sin embargo, en aplicaciones que requieren inserciones masivas y continuas como el registro de lecturas de sensores de temperatura a lo largo del tiempo, el árbol B clásico puede presentar un alto costo de escritura, ya que cada inserción puede desencadenar lecturas y escrituras costosas de bloques completos. Para abordar este problema, surgen los arboles B^ε, estructuras que extienden el árbol B incorporando buffers en sus nodos internos. Estos buffers permiten encolar las inserciones y aplicar un vaciado por lotes hacia los hijos, reduciendo asintóticamente el costo amortizado de escritura, a cambio de una leve penalización en búsquedas puntuales y un mayor uso de CPU.

El presente informe detalla el diseño, la implementación y el análisis experimental de la Tarea 2, cuyo objetivo principal es comparar el rendimiento práctico de un árbol B tradicional frente a un árbol B^ε (evaluado en dos configuraciones: $\varepsilon \approx 0.25$ y $\varepsilon \approx 0.75$). Para garantizar un entorno de prueba justo y apegado a la teoría de memoria externa, ambas estructuras fueron implementadas forzando un tamaño exacto de nodo de 4096 bytes.

A lo largo del experimento, se evalúan métricas críticas variando el volumen de datos de entrada desde $N = 2^{15}$ hasta $N = 2^{26}$ elementos. El análisis se divide en dos fases:

1. **Fase de Construcción:** Donde se mide el tiempo de inserción y se simulan/cuantifican las operaciones de lectura y escritura en memoria RAM.
2. **Fase de Consulta:** Donde, tras serializar los árboles a archivos binarios, se ejecutan búsquedas por rango (intervalos de una semana) leyendo directamente desde el disco para comparar el tiempo de respuesta y la cantidad de bloques transferidos.

Finalmente, este documento expone los resultados obtenidos mediante gráficos comparativos, discutiendo empíricamente si la ventaja asintótica de las escrituras del árbol B^ε se justifica frente a la sobrecarga administrativa de sus buffers en la práctica.

1.1. Hipótesis

Se plantea que el Árbol B tradicional presentará los tiempos de búsqueda por rango más rápidos y la menor cantidad de transferencias de bloques desde el disco, debido a que sus datos se encuentran totalmente organizados en las hojas sin elementos pendientes en niveles superiores. Sin embargo, registrará el mayor costo en operaciones de escritura (I/Os simuladas) y tiempo de ejecución durante la fase de inserción masiva.

Para las variantes del Árbol B^ε, se proyecta el siguiente comportamiento:

- **Configuración $\varepsilon \approx 0.25$ ($c = 4$, $l_{\text{máx}} = 506$):** Dominará en la fase de inserción, registrando la menor cantidad de escrituras en RAM debido a la gran capacidad de sus buffers internos (506 elementos). No obstante, sufrirá una degradación severa en los tiempos de búsqueda por rango, ya que cada consulta deberá inspeccionar buffers de gran tamaño en los nodos internos antes de llegar a las hojas.
- **Configuración $\varepsilon \approx 0.75$ ($c = 126$, $l_{\text{máx}} = 384$):** Exhibirá un rendimiento intermedio. Presentará búsquedas más rápidas que la versión de $\varepsilon \approx 0.25$ debido a su mayor grado de ramificación (127 hijos) y menor profundidad, pero requerirá un tiempo de construcción y un número de escrituras superior al vaciar sus buffers (384 elementos) con mayor frecuencia.

2. Desarrollo

2.1. Estructuras de Datos y Ajuste de Bloque

Para garantizar que cada nodo ocupe exactamente 4096 bytes tanto en RAM como en disco, se definieron estructuras C++ estrictamente empaquetadas (`#pragma pack(push, 1)`) utilizando tipos de datos de tamaño fijo:

- **Árbol B:** Cada nodo contiene los campos `es_interno` (4 bytes), `count` (4 bytes), un arreglo `items` de $b = 340$ pares llave-valor (2720 bytes), un arreglo `children` de 341 enteros (1364 bytes) y una variable de `padding` de 4 bytes (fijada en 0), alcanzando exactamente $4 + 4 + 2720 + 1364 + 4 = 4096$ bytes.
- **Árbol B^ε:** Se implementó mediante una clase plantilla (`template <size_t C, size_t LMAX>`) que parametriza la cantidad de llaves divisorias C y la capacidad del buffer LMAX. La estructura del nodo cumple con $8 \times (2 + C + \mathrm{LMAX}) = 4096$ bytes, forzando la restricción $C + \mathrm{LMAX} = 510$ para ambas configuraciones evaluadas ($c = 4, l_{\text{máx}} = 506$ y $c = 126, l_{\text{máx}} = 384$).

2.2. Decisiones de Implementación y Simulación de E/S

Dado que realizar millones de inserciones reales en disco llevaría un tiempo impráctico para la experimentación, se tomaron las siguientes decisiones de diseño no especificadas directamente en el enunciado:

1. **Abstracción de Lecturas/Escrituras:** El árbol se mantiene en memoria principal como un `std::vector` contiguo de nodos donde el índice 0 corresponde siempre a la raíz. En el árbol B se utilizan los helpers `readNode` y `writeNode` para contabilizar los accesos a nodos. En el árbol B^ε, los accesos se contabilizan explícitamente durante la inserción, los **split**, la propagación de hijos y el **flush**. Cada lectura o escritura de un nodo completo incrementa los contadores de E/S simulada.
2. **Evitación de Referencias Colgadas (Dangling References):** Al utilizar un `std::vector` dinámico que puede reasignar su memoria al crecer, se evitó mantener referencias o punteros C++ directos a los nodos durante las operaciones de **split** o **flush**. En su lugar, se trabaja exclusivamente con índices enteros dentro del vector, reobteniendo la referencia al nodo únicamente después de cualquier reasignación (`push_back`).

2.3. Inserción y división de nodos

En el árbol B, antes de descender hacia un hijo se verifica si este está lleno. Si contiene $b = 340$ pares, se divide en dos nodos y se promueve al padre el par mediano. El nodo izquierdo conserva los pares menores o iguales al separador y el nodo derecho los pares mayores. En los nodos internos también se redistribuyen los índices de los hijos. Cuando la raíz está llena, se copia a una nueva posición, se divide y se crea una nueva raíz en el índice 0.

En el árbol B^ε, las inserciones se agregan ordenadamente al buffer del nodo raíz. Cuando una hoja se llena, se divide aproximadamente por la mitad y se copia al padre la llave separadora, conservando la llave correspondiente en la hoja izquierda. Cuando un nodo interno alcanza su máximo de c llaves divisorias, se divide alrededor de su llave central, se redistribuyen sus hijos y se separan los elementos de su buffer según el nuevo separador. Si una división afecta a la raíz, se crea una nueva raíz en la posición 0.

La diferencia principal es el momento en que los pares se envían hacia las hojas. En el árbol B cada inserción desciende inmediatamente hasta el nodo correspondiente. En B^ε las inserciones permanecen temporalmente en buffers internos y se envían en lotes durante un `flush`, reduciendo potencialmente la cantidad amortizada de accesos por elemento.

2.4. Algoritmo de Vaciado de Buffers (Flush)

El vaciado de buffers es la operación central del Árbol B^ε. En nuestra implementación se adoptaron los siguientes criterios:

- **Criterio del Palomar y Umbral α:** Dado un nodo interno desbordado con k_{buffer} elementos y $k_{\text{llaves}} + 1$ hijos, se calcula la carga promedio $f = \left\lceil \frac{k_{\text{buffer}}}{k_{\text{llaves}}+1} \right\rceil$. Se seleccionan únicamente aquellos hijos que reciben al menos $\lceil f \times \alpha \rceil$ elementos, donde $\alpha = 0.5$ fue el valor base seleccionado para los experimentos principales.
- **Agrupamiento por Hijo:** Los elementos del buffer se clasifican en arreglos temporales asignados a cada hijo en una sola pasada $O(k_{\text{buffer}})$.
- **Procesamiento de Derecha a Izquierda:** La transferencia de elementos hacia los hijos identificados se realiza recorriendo los índices de los hijos de derecha a izquierda. Esta decisión previene errores de desfasamiento en los índices de los subárboles en caso de que uno de los hijos sufra un **split** (división) durante el vaciado.

Desde el punto de vista amortizado, un `flush` procesa simultáneamente un grupo de pares en lugar de ejecutar una transferencia independiente por cada inserción. Si un buffer contiene m elementos, el agrupamiento cuesta $O(m)$ y la transferencia depende de la cantidad de hijos afectados y de los posibles **split**. Este costo se reparte entre los m elementos procesados, por lo que el costo promedio por inserción puede ser menor que en un árbol B tradicional. Sin embargo, esta ventaja asintótica no implica necesariamente menor tiempo de CPU: ordenar buffers, crear vectores temporales y buscar padres también introduce trabajo adicional.

En términos de accesos a bloques, el árbol B puede requerir un recorrido de altura h_B por inserción, con costo del orden de $O(\log_B N)$. En un árbol B^ε, los pares se acumulan y se amortiza el costo de propagar un lote sobre sus elementos. La formulación asintótica usual expresa este costo como del orden de $O\left(\frac{\log_{B^\varepsilon} N}{B^{1-\varepsilon}}\right)$ accesos amortizados por inserción, ignorando constantes y el costo de los **split**. Esta expresión explica la motivación de los buffers, mientras que las mediciones muestran el efecto de los costos adicionales de la implementación concreta.

El parámetro α controla qué grupos se transfieren durante cada `flush`. Un umbral bajo procesa más grupos, mientras que uno alto deja más elementos pendientes. Por ello, α afecta simultáneamente la frecuencia de los vaciados, el tamaño de los lotes, las escrituras simuladas y la cantidad de buffers que deben inspeccionarse durante una búsqueda.

2.5. Altura y efecto de ε

La altura depende de la cantidad de hijos que puede tener cada nodo interno. Para el árbol B, la ramificación es del orden de B, con $b = 340$, por lo que:

$$h_B = O(\log_B N). \tag{1}$$

En un árbol B^ε, la ramificación interna es del orden de B^ε, por lo que:

$$h_{B^\varepsilon} = O(\log_{B^\varepsilon} N). \tag{2}$$

En nuestras configuraciones, $B^\varepsilon_{\{0.25\}}$ utiliza $c + 1 = 5$ hijos por nodo interno, mientras que $B^\varepsilon_{\{0.75\}}$ utiliza $c + 1 = 127$. Por lo tanto, se espera que $B^\varepsilon_{\{0.75\}}$ tenga menor altura que $B^\varepsilon_{\{0.25\}}$. Esto ayuda a explicar que $B^\varepsilon_{\{0.75\}}$ obtuviera mejores tiempos de búsqueda que $B^\varepsilon_{\{0.25\}}$ para los tamaños grandes, aun cuando su buffer es más pequeño y registra más escrituras.

2.6. Búsquedas por Rango sobre Archivo Binario

Una vez construidos en RAM, los árboles se serializan escribiendo sus nodos de forma secuencial en un archivo `.bin`. Las consultas por rango $[l, u]$ de una semana (604.800 segundos) leen directamente desde el archivo binario:

- **Acceso Directo por Desplazamiento:** Para acceder al nodo i, el programa realiza un desplazamiento de archivo (`seekg`) a la posición $i \times 4096$ bytes y lee un único bloque.
- **Filtrado y Podado de Ramas en Árbol B:** En los nodos internos, la búsqueda no explora todos los hijos indiscriminadamente; se evalúan las llaves pivote para descender únicamente a los subárboles cuyos rangos de llaves intersectan con el intervalo $[l, u]$.
- **Inspección de Buffers en Árbol B^ε:** Dado que en el Árbol B^ε los datos pueden encontrarse en tránsito en cualquier nivel, la búsqueda examina el buffer de cada nodo interno visitado recolectando las llaves dentro del rango $[l, u]$, para luego continuar recursivamente hacia los hijos cuyas llaves divisorias traslapen el intervalo buscado.

3. Resultados

3.1. Especificaciones del Experimento

Los experimentos se ejecutaron utilizando el dataset de mediciones de temperatura de la Estación Quinta Normal (enero 2019 a julio 2025). Las pruebas de inserción evaluaron volúmenes de datos N desde 2^{15} hasta 2^{26} elementos. Para la fase de consulta, se realizaron 50 búsquedas por rango aleatorias por cada árbol y cada tamaño N, utilizando ventanas de tiempo de 604.800 segundos (equivalentes a una semana).

En cada caso se utilizaron los primeros N pares de `datos.bin`: una llave entera correspondiente a un timestamp Unix y un valor `float` correspondiente a la temperatura. Los tamaños evaluados fueron $N \in \{2^{15}, 2^{16}, \ldots, 2^{26}\}$. Para cada árbol se midieron el tiempo de construcción, las lecturas y escrituras simuladas durante la inserción, la cantidad de nodos, el tiempo promedio de búsqueda y las lecturas promedio de bloques.

Los límites inferiores se generaron con `mt19937`, semilla 42 y distribución uniforme en:

$$[1546300800, 1754006400 - 604800]. \tag{3}$$

Para cada consulta se definió $u = l + 604800$. El generador se comparte entre los bloques de experimentación, por lo que no necesariamente se utiliza exactamente la misma secuencia de 50 rangos para los tres árboles; las diferencias pequeñas entre tiempos deben interpretarse con cautela. El experimento adicional de α utilizó $N = 2^{24}$, la configuración $B^{\varepsilon}_{\{0.25\}}$, ocho valores de α entre 0.1 y 0.8, 20 búsquedas por valor y semilla 4242.

3.1.1. Setup Experimental

Parámetro	Valor
CPU	AMD Ryzen 5 3600, 4.2 GHz
Caché (L1 / L2 / L3)	384 KB / 3 MB / 32 MB
RAM	16 GB DDR4 3200 MHz
OS	Arch Linux

Tabla 1: Especificaciones de pruebas y de compilación.

3.2. Costo de Inserción y Construcción

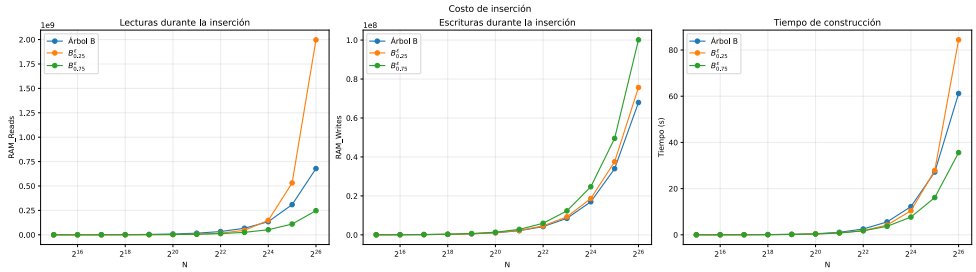


Figura 1: Lecturas, escrituras y tiempo de construcción en función de N.

En la figura 1 se observa que el Árbol B clásico registró la menor cantidad de escrituras en RAM para los tamaños grandes. Sin embargo, no fue la estructura más rápida en tiempo de construcción. Para $N = 2^{\{26\}}$, la configuración $B^{\varepsilon}_{\{0.75\}}$ obtuvo el menor tiempo, con aproximadamente 35.6 segundos; el Árbol B tardó 61.2 segundos y $B^{\varepsilon}_{\{0.25\}}$ tardó 84.4 segundos. La variante $B^{\varepsilon}_{\{0.75\}}$ presentó el mayor volumen de escrituras en RAM, con aproximadamente 100.1 millones de escrituras.

3.3. Costos de Búsqueda por Rango

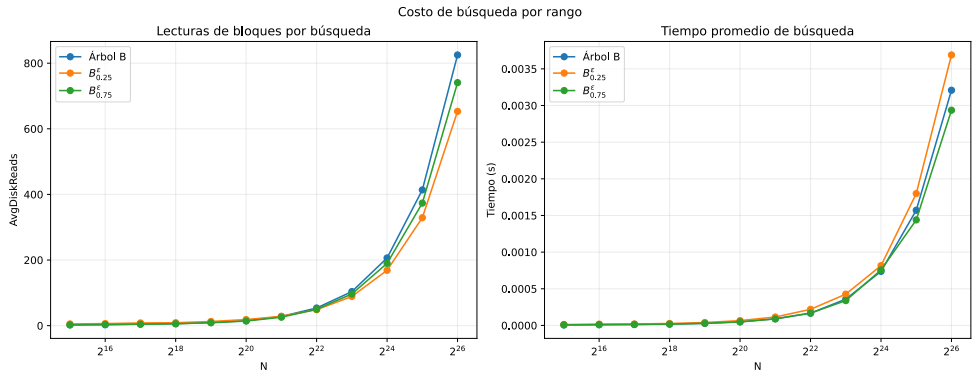


Figura 2: Lecturas de bloques y tiempo promedio de búsqueda por rango.

La figura 2 muestra que el Árbol B no fue consistentemente la estructura más rápida en tiempo de CPU. Para $N = 2^{\{26\}}$, $B^{\varepsilon}_{\{0.75\}}$ obtuvo el menor tiempo promedio, con aproximadamente 0.00294 segundos, seguido por el Árbol B con 0.00321 segundos y $B^{\varepsilon}_{\{0.25\}}$ con 0.00369 segundos. En cuanto a lecturas de bloques, $B^{\varepsilon}_{\{0.25\}}$ utilizó menos lecturas que el Árbol B para los tamaños grandes, aunque no para todos los valores de N.

3.4. Tamaño de los árboles

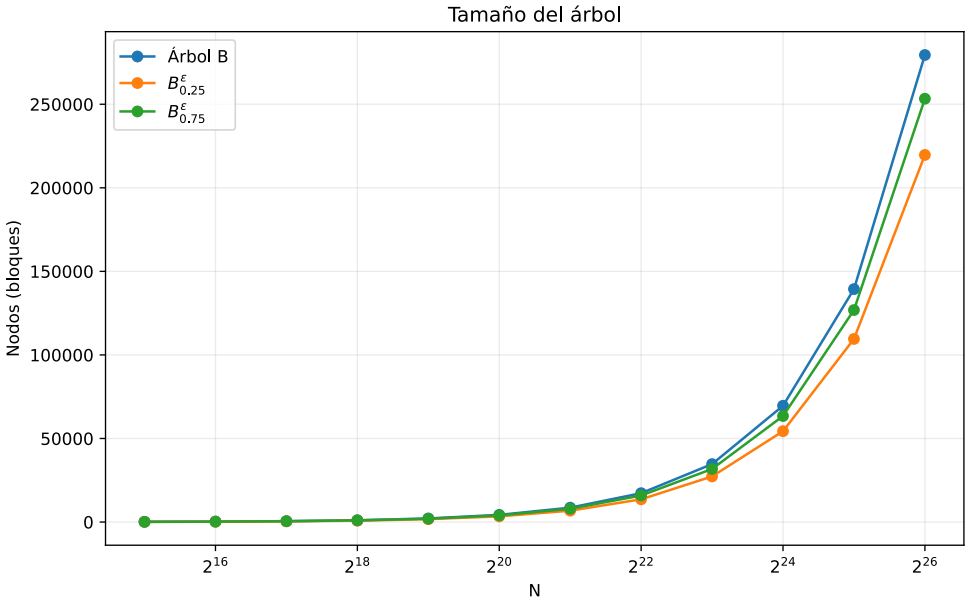


Figura 3: Cantidad de nodos, equivalentes a bloques de 4096 bytes, en función de N.

La figura 3 muestra que $B^{\varepsilon}_{\{0.25\}}$ utiliza menos nodos que las otras estructuras para todos los tamaños evaluados. Para $N = 2^{\{26\}}$, utiliza 219.664 nodos, mientras que el Árbol B utiliza 279.471 y $B^{\varepsilon}_{\{0.75\}}$ utiliza 253.304. Esto se explica por la mayor capacidad de almacenamiento de sus buffers y por la menor cantidad de hijos divisores de esta configuración.

3.5. Comparación de una misma búsqueda

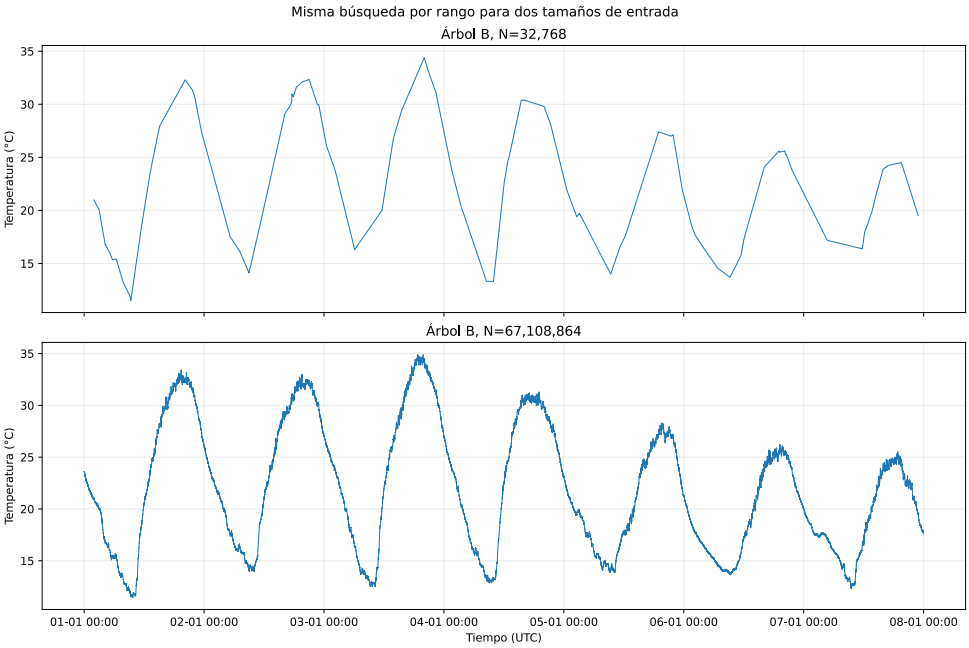


Figura 4: Temperaturas obtenidas en una misma búsqueda por rango para $N = 2^{15}$ y $N = 2^{26}$.

La figura 4 muestra los resultados de una búsqueda fija de una semana para dos tamaños de entrada. El intervalo consultado es el mismo, pero los conjuntos no son idénticos porque cada árbol contiene un

prefijo de distinto tamaño del dataset. Para $N = 2^{\{15\}}$ se obtuvieron 97 mediciones, mientras que para $N = 2^{\{26\}}$ se obtuvieron 195.318 mediciones. La diferencia se debe a que el segundo árbol contiene muchos más pares correspondientes al intervalo consultado.

3.6. Resultados del Parámetro α

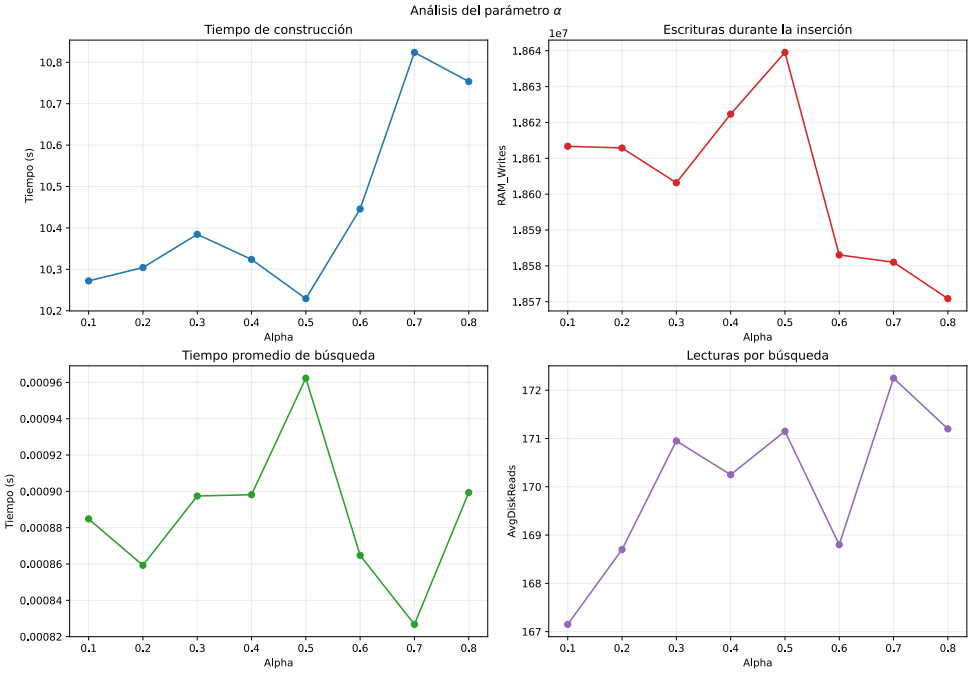


Figura 5: Impacto de α sobre el tiempo de construcción, las escrituras de inserción, el tiempo de búsqueda y las lecturas promedio por búsqueda.

Al fijar $N = 2^{\{24\}}$ y variar el parámetro de agresividad de vaciado α entre 0.1 y 0.8, la figura 5 no muestra una tendencia monótona clara. La menor cantidad de lecturas promedio se obtuvo con $\alpha = 0.1$, con 167.15 lecturas, mientras que el menor tiempo promedio de búsqueda se obtuvo con $\alpha = 0.7$, con aproximadamente 0.000827 segundos. Las diferencias entre configuraciones son pequeñas, por lo que no es posible identificar un único valor óptimo para todas las métricas con estas 20 consultas por configuración.

4. Análisis

El comportamiento observado durante la fase de construcción difiere de la ventaja teórica de las inserciones por lotes del Árbol B^ε. Aunque el Árbol B clásico registró menos escrituras simuladas, la configuración $B^\varepsilon_{\{0.75\}}$ obtuvo el menor tiempo de construcción para $N = 2^{\{26\}}$. En cambio, $B^\varepsilon_{\{0.25\}}$ fue la configuración más lenta, lo que sugiere que la sobrecarga de gestionar, ordenar y vaciar (**flush**) sus buffers grandes penalizó el tiempo de CPU. $B^\varepsilon_{\{0.75\}}$ registró más escrituras debido a su buffer reducido ($l_{\text{máx}} = 384$), pero su mayor ramificación pudo compensar ese costo en tiempo total.

En la fase de consulta, el Árbol B no domina siempre en tiempo de CPU: para $N = 2^{\{26\}}$, $B^\varepsilon_{\{0.75\}}$ obtuvo el menor tiempo promedio. Las variantes B^ε deben inspeccionar elementos pendientes en los buffers de los nodos internos, pero su menor altura puede compensar parte de ese costo. Además, $B^\varepsilon_{\{0.25\}}$ obtuvo menos lecturas de bloques que el Árbol B para los tamaños grandes, aunque esta ventaja no aparece en los tamaños pequeños.

4.1. Conclusión

La implementación permitió comparar un Árbol B y dos configuraciones de árboles B^ε utilizando nodos de 4096 bytes, inserciones en un arreglo que simula memoria externa y búsquedas realizadas sobre archivos binarios serializados. Los resultados muestran que no existe una estructura superior en todas las métricas.

El Árbol B utilizó menos escrituras simuladas que las variantes B^ε, pero no obtuvo el menor tiempo de construcción para el tamaño máximo. En $N = 2^{\{26\}}$, $B^\varepsilon_{\{0.75\}}$ fue la configuración más rápida para construir y también obtuvo el menor tiempo promedio de búsqueda. Por otro lado, $B^\varepsilon_{\{0.25\}}$ produjo los árboles más pequeños y, para los tamaños grandes, requirió menos lecturas de bloques durante las búsquedas.

La hipótesis inicial se cumplió solo parcialmente. Se esperaba que el Árbol B tuviera búsquedas más rápidas, pero los resultados favorecieron a $B^\varepsilon_{\{0.75\}}$ en el tamaño máximo. También se confirmó que los buffers modifican el compromiso entre costo de construcción, cantidad de escrituras, tamaño del árbol y costo de consulta. El análisis de α no identificó un valor óptimo único, debido a que las diferencias entre configuraciones fueron pequeñas y se realizaron solo 20 consultas por valor.

Como trabajo futuro, sería conveniente repetir cada configuración varias veces, utilizar exactamente las mismas consultas aleatorias para todos los árboles y reportar intervalos de confianza. En este trabajo no se midieron por separado las alturas exactas, la distribución de elementos entre buffers ni la cantidad de resultados provenientes de cada nivel. Tampoco se aisló completamente el efecto de las cachés del sistema operativo ni se separó el costo de CPU de cada operación interna del `flush`. Estas son limitaciones que podrían abordarse en una extensión, junto con la evaluación de más valores de α alrededor de los mejores candidatos.

Finalmente, la evaluación del parámetro α confirma que existe un compromiso entre las métricas, aunque no se observa una tendencia monotónica con los ocho valores probados. El menor número de lecturas promedio se obtuvo con $\alpha = 0.1$, mientras que el menor tiempo promedio de búsqueda se obtuvo con $\alpha = 0.7$. Por ello, la elección de α debe depender de si se prioriza el costo de construcción o el costo de consulta.